6 Implementing authentications

This chapter covers

- Implementing authentication logic using a custom `AuthenticationProvider`
- Using the HTTP Basic and form-based login authentication methods
- Understanding and managing the `Security-Context` component

Chapters 3 and 4 covered a few of the components acting in the authentication flow. We discussed `UserDetails` and how to define the prototype to describe a user in Spring Security. Next, we used `UserDetails` in examples that showed how the `UserDetailsService` and `UserDetailsManager` contracts work and how to implement them. We discussed and used the leading implementations of these interfaces in examples as well. Finally, you learned how a `PasswordEncoder` manages passwords and how to use one, as well as the Spring Security crypto module (SSCM) with its encryptors and key generators.

The `AuthenticationProvider` layer, however, is responsible for the logic of authentication. The `AuthenticationProvider` is where you find the conditions and instructions that decide whether to authenticate a request. The component that delegates this responsibility to the `AuthenticationProvider` is the `AuthenticationManager`, which receives the request from the HTTP filter layer, and it was discussed in chapter 5. In this chapter, we'll look at the authentication process, which has only two possible results:

- *The entity making the request is not authenticated.* The user is not recognized, and the application rejects the request without delegating to the authorization process. Usually, the response status sent back to the client in this case is HTTP 401 Unauthorized.
- *The entity making the request is authenticated.* The details about the requester are stored so that the application can use them for authorization. As you'll find out in this chapter, `SecurityContext` is responsible for the details regarding the current authenticated request.

To remind you of the actors and the links between them, figure 6.1 shows the diagram that we already saw in chapter 2.

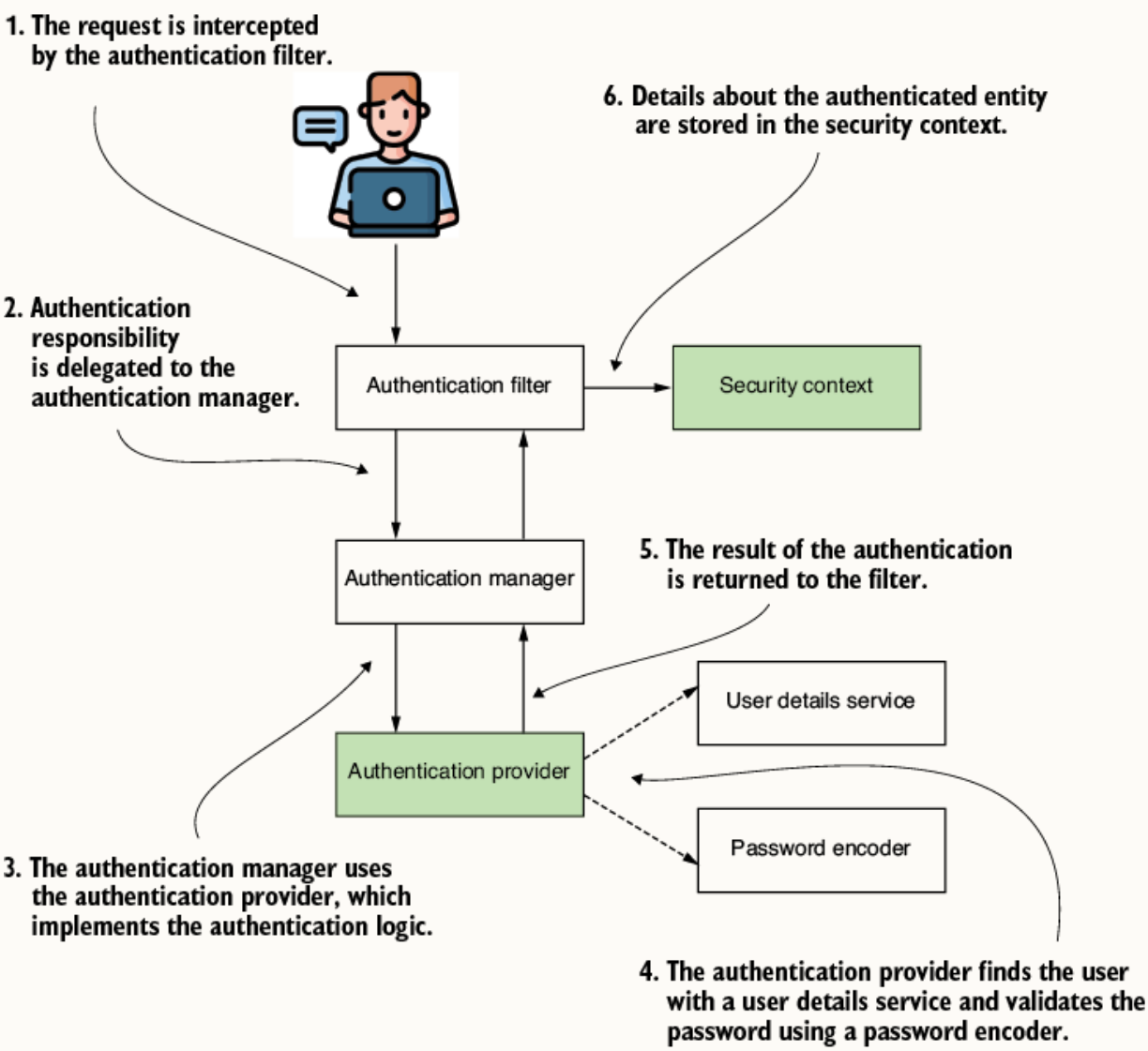


Figure 6.1 The authentication flow in Spring Security. This process outlines the method by which the application recognizes the individual submitting a request. The elements that are the focus of this chapter are highlighted. In this context, the `AuthenticationProvider` is responsible for carrying out the authentication procedure, and the `SecurityContext` retains the information regarding the request that has been authenticated.

This chapter covers the remaining parts of the authentication flow (the shaded boxes in figure 6.1). Then, in chapters 7 and 8, you'll learn how authorization works, which is the process that follows authentication in the HTTP request. First, we need to discuss how to implement the `AuthenticationProvider` interface. You need to know how Spring Security understands a request in the authentication process.

To show a clear description of how to represent an authentication request, we'll start with the `Authentication` interface. Once we discuss it, we can go further and observe what happens with the details of a request after successful authentication. Then we can discuss the `SecurityContext` interface and the way Spring Security manages it. Near the end of the chapter, you'll learn how to customize the HTTP Basic authentication method. We'll also discuss another option for authentication that can be used in our applications—the form-based login.

6.1 Understanding the AuthenticationProvider

In enterprise applications, you might find yourself in a situation in which the default implementation of authentication based on username and password does not apply. In addition, when it comes to authentication, your application may require the implementation of several scenarios (figure 6.2). For example, you might want the user to be able to prove who they are by using a code received in an SMS message or displayed by a specific application. Or you might need to implement authentication scenarios where the user has to provide a certain kind of key stored in a file. You might even need to use a representation of the user's fingerprint to implement the authentication logic. A framework's purpose is to be flexible enough to allow you to implement any of these scenarios.

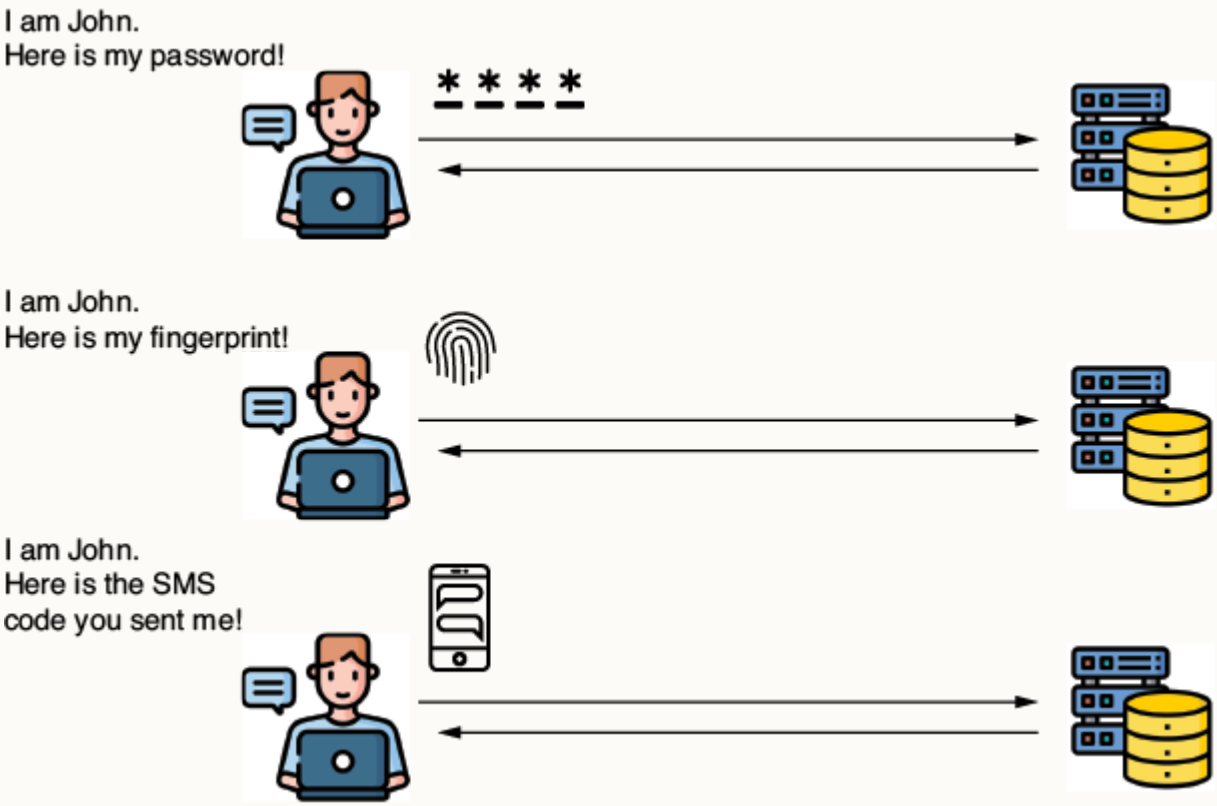


Figure 6.2 An application may require various methods of authentication implementation. Although a username and password suffice for most situations, there are instances where the process of authenticating a user may be more complex.

A framework usually provides a set of the most used implementations, but of course, it cannot cover all the possible options. In terms of Spring Security, you can use the `AuthenticationProvider` contract to define any custom authentication logic. In this section, you'll learn to represent the authentication event by implementing the `Authentication` interface and then create your custom authentication logic with an `AuthenticationProvider`. To achieve our goal

- In section 6.1.1, we analyze how Spring Security represents the authentication event.

- In section 6.1.2, we discuss the `AuthenticationProvider` contract, which is responsible for the authentication logic.
- In section 6.1.3, you'll write custom authentication logic by implementing the `AuthenticationProvider` contract in an example.

6.1.1 Representing the request during authentication

This section discusses how Spring Security understands a request during the authentication process. It is important to touch on this before diving into implementing custom authentication logic. As you'll learn in section 6.1.2, to implement a custom `AuthenticationProvider`, you first need to understand how to describe the authentication event. Here, we'll take a look at the contract representing authentication and discuss the methods you need to know.

`Authentication` is one of the essential interfaces involved in the process with the same name. The `Authentication` interface represents the authentication request event and holds the details of the entity that requests access to the application. You can use the information related to the authentication request event during and after the authentication process. The user requesting access to the application is called a *principal*. If you've ever used Java Security in any app, you probably know that an interface named `Principal` represents the same concept. The `Authentication` interface of Spring Security extends this contract (figure 6.3).

Figure 6.3 The `Authentication` agreement extends the `Principal` agreement. It introduces additional stipulations, such as the necessity for a password or the option to provide further specifics regarding the authentication request. Certain aspects, such as the array of authorities, are specific to Spring Security.

The `Authentication` contract in Spring Security not only represents a principal, but it also adds information on whether the authentication process finishes, as well as a collection of authorities. The fact that this contract was designed to extend the `Principal` contract from Java Security is a plus in terms of compatibility with implementations of other frameworks and applications. This flexibility allows for more facile migrations to Spring Security from applications that implement authentication in another fashion.

Let's find out more about the design of the `Authentication` interface in the following listing.

Listing 6.1 The `Authentication` interface as declared in Spring Security

```
public interface Authentication extends Principal, Serializable {

  Collection<? extends GrantedAuthority> getAuthorities();
  Object getCredentials();
  Object getDetails();
  Object getPrincipal();
  boolean isAuthenticated();
  void setAuthenticated(boolean isAuthenticated)
    throws IllegalArgumentException;
}
```

For the moment, the only methods of this contract that you need to learn are these:

- `isAuthenticated()` —Returns true if the authentication process ends or false if the authentication process is still in progress
- `getCredentials()` —Returns a password or any secret used in the authentication process
- `getAuthorities()` —Returns a collection of granted authorities for the authenticated request

We'll discuss the other methods for the `Authentication` contract in later chapters, where appropriate for the implementations considered.

6.1.2 Implementing custom authentication logic

This section deals with implementing custom authentication logic. We analyze the Spring Security contract related to this responsibility to understand its definition. With these details, you can implement custom authentication logic with a code example in section 6.1.3.

The `AuthenticationProvider` in Spring Security takes care of the authentication logic. The default implementation of the `AuthenticationProvider` interface delegates the responsibility of finding the system's user to a `UserDetailsService`. It uses the `PasswordEncoder` as well for password management in the authentication process. The following listing gives the definition of the `AuthenticationProvider`, which you need to define a custom authentication provider for your application.

Listing 6.2 The `AuthenticationProvider` interface

```
public interface AuthenticationProvider {
```

```
    Authentication authenticate(Authentication authentication)
     throws AuthenticationException;

    boolean supports(Class<?> authentication);
}
```

The `AuthenticationProvider` responsibility is strongly coupled with the `Authentication` contract. The `authenticate()` method receives an `Authentication` object as a parameter and returns an `Authentication` object. We implement the `authenticate()` method to define the authentication logic. Here, we quickly summarize the way you should implement the `authenticate()` method:

- The method should throw an `AuthenticationException` if the authentication fails.
- If the method receives an authentication object not supported by your implementation of `AuthenticationProvider`, then the method should return null. This way, we have the possibility of using multiple `Authentication` types separated at the HTTP-filter level.
- The method should return an `Authentication` instance representing a fully authenticated object. For this instance, the `isAuthenticated()` method returns true, and it contains all the necessary details about the authenticated entity. Usually, the application also removes sensitive data, such as a password from this instance. After a successful authentication, the password is no longer required, and keeping these details can potentially expose them to unwanted eyes.

The second method in the `AuthenticationProvider` interface is `supports(Class<?> authentication)`. You can implement this method to return true if the current `AuthenticationProvider` supports the type provided as an `Authentication` object. Observe that even if this method returns true for an object, there is still a chance that the `authenticate()` method will reject the request by returning null. Spring Security is designed to be more flexible, allowing users to implement an `AuthenticationProvider` that can reject an authentication request based on its details, and not only based on its type.

An analogy of how the authentication manager and authentication provider work together to validate or invalidate an authentication request is having a more complex lock for your door. You can open this lock either by using a card or an old-fashioned physical key (figure 6.4). The lock itself is the authentication manager that decides whether to open the door. To make that decision, it delegates to the two authentication providers: one that knows how to validate the card or the other that knows how to verify the physical key. If you present a card to open the door, the authentication provider that works

only with physical keys complains that it is not familiar with this kind of authentication. However, the other provider supports this kind of authentication and verifies whether the card is valid for the door. This is the purpose of the `supports()` methods.

In addition to testing the authentication type, Spring Security adds one more layer for flexibility. The door's lock can recognize multiple kinds of cards. In this case, when you present a card, one of the authentication providers could say, "I understand this as being a card. But it isn't the type of card I can validate!" This happens when `supports()` returns true but `authenticate()` returns null.

Figure 6.4 The `AuthenticationManager` delegates to one of the available authentication providers. The `AuthenticationProvider` might not support the provided authentication type. However, if it does support the object type, it might not know how to authenticate that specific object. The authentication is evaluated, and an `AuthenticationProvider` that can say whether or not the request is correct responds to the `AuthenticationManager`.

Figure 6.5 shows the alternative scenario where one of the `AuthenticationProvider` objects recognizes the `Authentication` but decides it's not valid. In this case, the result will be an `AuthenticationException` that ends up as a 401 Unauthorized HTTP status in the HTTP response in a web app.

Figure 6.5 If none of the `AuthenticationProvider` objects recognize the `Authentication` or any of them rejects it, the result is an `AuthenticationException`.

6.1.3 Applying custom authentication logic

In this section, we implement custom authentication logic. You can find this example in the project ssia-ch6-ex1. With this example, you apply what you've learned about the `Authentication` and `AuthenticationProvider` interfaces in sections 6.1.1 and 6.1.2. In listings 6.3 and 6.4, we build an example of how to implement a custom `AuthenticationProvider`. These steps, also presented in figure 6.5, are as follows:

1. Declare a class that implements the `AuthenticationProvider` contract.
2. Decide which kinds of `Authentication` objects are supported by the new `AuthenticationProvider`.
3. Implement the `supports(Class<?> c)` method to specify which type of authentication is supported by the `AuthenticationProvider` that we define.
4. Implement the `authenticate(Authentication a)` method to implement the authentication logic.

5. Register an instance of the new `AuthenticationProvider` implementation with Spring Security.

Listing 6.3 Overriding the `supports()` method of the `AuthenticationProvider`

```
@Component
public class CustomAuthenticationProvider
 implements AuthenticationProvider {

 // Omitted code

 @Override
 public boolean supports(Class<?> authenticationType) {
  return authenticationType
        .equals(UsernamePasswordAuthenticationToken.class);
 }
}
```

In listing 6.3, we define a new class that implements the `AuthenticationProvider` interface. We mark the class with `@Component` to have an instance of its type in the context managed by Spring. Then we must decide what kind of `Authentication` interface implementation this `AuthenticationProvider` supports. That depends on what type we expect to be provided as a parameter to the `authenticate()` method. If we don't customize anything at the authentication filter level (as discussed in chapter 5), then the class `UsernamePasswordAuthenticationToken` defines the type. This class is an implementation of the `Authentication` interface and represents a standard authentication request with username and password.

With this definition, we made the `AuthenticationProvider` support a specific kind of key. Once we have specified the scope of our `AuthenticationProvider`, we implement the authentication logic by overriding the `authenticate()` method, as shown in following listing.

Listing 6.4 Implementing the authentication logic

```
@Component
public class CustomAuthenticationProvider
 implements AuthenticationProvider {

 private final UserDetailsService userDetailsService;
 private final PasswordEncoder passwordEncoder;
```

```
    // Omitted constructor

    @Override
    public Authentication authenticate(Authentication authentication) {
     String username = authentication.getName();
     String password = authentication.getCredentials().toString();

     UserDetails u = userDetailsService.loadUserByUsername(username);

     if (passwordEncoder.matches(password, u.getPassword())) {
      return new UsernamePasswordAuthenticationToken(
          username,
          password,
          u.getAuthorities());                ①
     } else {
      throw new BadCredentialsException
              ("Something went wrong!");      ②
     }
    }

    // Omitted code
}
```

① If the password matches, it returns an implementation of the Authentication contract with the necessary details.

② If the password doesn't match, it throws an exception of type AuthenticationException. BadCredentialsException inherits from AuthenticationException.

The logic in listing 6.4 is simple, and figure 6.6 provides a visual presentation of this logic. We use the `UserDetailsService` implementation to get the `UserDetails`. If the user doesn't exist, the `loadUserByUsername()` method should throw an `AuthenticationException`. In this case, the authentication process stops, and the HTTP filter sets the response status to HTTP 401 Unauthorized. If the username does exist, we can further check the user's password with the `matches()` method of the `PasswordEncoder` from the context. If the password does not match, then again, an `AuthenticationException` should be thrown. If the password is correct, the `AuthenticationProvider` returns an instance of `Authentication` marked as "authenticated," which contains the request details.

Figure 6.6 The `AuthenticationProvider` enacts a tailored authentication procedure. It confirms the authentication request by retrieving user details through a specific `UserDetailsService` implementation, and it validates the password using a `PasswordEncoder` if the password is correct. If the user is not found or the password is inaccurate, the `AuthenticationProvider` will raise an `AuthenticationException`.

To plug in the new implementation of the `AuthenticationProvider`, we define a `SecurityFilterChain` bean. This is demonstrated in the following listing.

Listing 6.5 Registering the `AuthenticationProvider` in the configuration class

```
@Configuration
public class ProjectConfig {

 private final AuthenticationProvider authenticationProvider;

 // Omitted constructor

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.httpBasic(Customizer.withDefaults());

  http.authenticationProvider(authenticationProvider);

  http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

  return http.build();
 }

 // Omitted code
}
```

NOTE In listing 6.5, the dependency injection is employed with a field declared using the `AuthenticationProvider` interface. Spring recognizes the `AuthenticationProvider` as an interface (which is an abstraction). However, Spring knows that it needs to find an instance of an implementation in its context for that specific interface. In our case, the implementation is the instance of `CustomAuthenticationProvider`, which is the only one of this type that we declared and added to the Spring context using the `@Component` annotation. For a refresher on dependency injection, I recommend *Spring Start Here* (Manning, 2021), another book I wrote.

That's it! You successfully customized the implementation of the `AuthenticationProvider`. You can now customize the authentication logic for your application where you need it.

How to fail in application design

Applying a framework incorrectly leads to a less maintainable application. What is worse, sometimes those who fail in using the framework believe that it's the framework's fault. Let me tell you a story.

One winter, the head of development in a company I worked with as a consultant called me to help them with the implementation of a new feature. They needed to apply a custom authentication method in a component of their system developed with Spring in its early days. Unfortunately, when implementing the application's class design, the developers didn't rely properly on Spring Security's backbone architecture.

They only used the filter chain, reimplementing entire features from Spring Security as custom code.

Developers observed that with time, customizations became increasingly difficult. However, nobody took action to redesign the component properly and use the contracts as intended in Spring Security. Much of the difficulty came from not knowing Spring's capabilities. One of the lead developers said, "It's only the fault of this Spring Security! This framework is hard to apply, and it's difficult to use it with any customization." I was a bit shocked at his observation. I know that Spring Security is sometimes difficult to understand and that the framework is known for not having a soft learning curve. But I've never experienced a situation in which I couldn't find a way to design an easy-to-customize class with Spring Security!

We investigated the problem together, and I realized the application developers used perhaps only 10% of what Spring Security has to offer. Then I presented a two-day workshop on Spring Security, focusing on what we could do for the specific system component they had to change and how to do it.

Everything ended with the decision to completely rewrite a lot of custom code to rely on Spring Security correctly and thus make the application easier to extend to meet their concerns for security implementations. We also discovered some other problems unrelated to Spring Security, but that's another story.

Here are a few lessons for you to take from this story:

- A framework, and especially the one widely used in applications, is written by many smart individuals, and it's hard to believe that it can be implemented badly. Always analyze your application before concluding that any problem may be the framework's fault.
- When deciding to use a framework, make sure you understand, at least, its basics.
- Be mindful of the resources you use to learn about the framework. Sometimes, articles you find on the web show how to do quick workarounds and not necessarily how to correctly implement a class design.
- Use multiple sources in your research. To clarify your misunderstandings, write a proof of concept when unsure how to use something.
- If you decide to use a framework, use it as much as possible for its intended purpose. For example, say you use Spring Security, and you observe that for security implementations, you tend to write more custom code instead of relying on what the framework offers. You should question why this happens.

When we rely on functionalities implemented by a framework, we enjoy several benefits. We know they are tested, and there are fewer changes that include vulnerabilities. Likewise, a good framework relies on abstractions, which help you create maintainable applications. Remember that when you write your own implementations, you're more susceptible to including vulnerabilities.

6.2 Using the SecurityContext

This section discusses the security context. We analyze how it works, how to access data, and how the application manages it in different thread-related scenarios. Once you finish this section, you'll know how to configure the security context for various situations. This way, you can use the details about the authenticated user stored by the security context in configuring authorization in chapters 7 and 8.

It is likely that you will need details about the authenticated entity following the authentication process. You might, for example, need to refer to the username or the authorities of the currently authenticated user. Is this information still accessible after the authentication process finishes? Once the `AuthenticationManager` completes the authentication process successfully, it stores the `Authentication` instance for the rest of the request (see figure 6.7). The instance storing the `Authentication` object is called the *security context*.

Figure 6.7 After successful authentication, the authentication filter stores the details of the authenticated entity in the security context. From there, the controller implementing the action mapped to the request can access these details when needed.

The security context of Spring Security is described by the `SecurityContext` interface and defined in the following listing.

Listing 6.6 The `SecurityContext` interface

```
public interface SecurityContext extends Serializable {

  Authentication getAuthentication();
  void setAuthentication(Authentication authentication);
}
```

As you can tell from the contract definition, the primary responsibility of the `SecurityContext` is to store the `Authentication` object. But how is the `SecurityContext` itself managed? Spring Security offers three strategies to manage the `SecurityContext` with an object in the role of a manager. It's named the `SecurityContextHolder`:

- `MODE_THREADLOCAL`—Allows each thread to store its own details in the security context. In a thread-per-request web application, this is a common approach, as each request has an individual thread.
- `MODE_INHERITABLETHREADLOCAL`—Similar to `MODE_THREADLOCAL`, but it also instructs Spring Security to copy the security context to the next thread in case of an asynchronous method. This way, we can say that the new thread running the `@Async` method inherits the security context. The `@Async` annotation is used with methods to instruct Spring to call the annotated method on a separate thread.
- `MODE_GLOBAL`—Makes all the threads of the application see the same security context instance.

In addition to these three strategies for managing the security context provided by Spring Security, this section also illustrates what happens when you define your own threads that are not known by Spring. As you will learn, for these cases, you need to explicitly copy the details from the security context to the new thread. Spring Security cannot automatically manage objects that are not in Spring's context, but it offers some great utility classes.

6.2.1 Using a holding strategy for the security context

The first strategy for managing the security context is the `MODE_THREADLOCAL` strategy, and it is also the default for managing the security context used by Spring Security. With this strategy, Spring Security uses `ThreadLocal` to manage the context. `ThreadLocal` is an implementation provided by the JDK. This implementation works as a

collection of data but ensures that each thread of the application can only see the data stored in its dedicated part of the collection. This way, each request has access to its security context. No thread will have access to another's `ThreadLocal`. That means that in a web application, each request can see only its own security context. We could say that this is also what you generally want to have for a backend web application.

Figure 6.8 offers an overview of this functionality. Each request (A, B, and C) has its own allocated thread (T1, T2, and T3), so each request only sees the details stored in its own security context. However, this also means that if a new thread is created (for example, when an asynchronous method is called), the new thread will have its own security context as well. The details from the parent thread (the original thread of the request) are not copied to the security context of the new thread.

NOTE Here we discuss a traditional servlet application where each request is tied to a thread. This architecture only applies to the traditional servlet application where each request has its own thread assigned. It does not apply to reactive applications. We'll discuss the security for reactive approaches in detail in chapter 17.

Being the default strategy for managing the security context, this process does not need to be explicitly configured. Just ask for the security context from the holder using the static `getContext()` method wherever you need it after the end of the authentication process. In listing 6.7, you find an example of obtaining the security context in one of the application endpoints. From the security context, you can further get the `Authentication` object, which stores the details about the authenticated entity. You can find the examples discussed in this section as part of the project ssia-ch6-ex2.

Figure 6.8 Each request has its own thread represented by an arrow. Each thread has access only to its own security context details. When a new thread is created (for example, by an `@Async` method), the details from the parent thread aren't copied.

Listing 6.7 Obtaining the `SecurityContext` from the `SecurityContextHolder`

```
@GetMapping("/hello")
public String hello() {
  SecurityContext context = SecurityContextHolder.getContext();
  Authentication a = context.getAuthentication();

  return "Hello, " + a.getName() + "!";
}
```

Obtaining the authentication from the context is even more comfortable at the endpoint level, as Spring knows to inject it directly into the method parameters. You don't need to refer every time to the `SecurityContextHolder` class explicitly. This approach, as presented in the following listing, is better.

Listing 6.8 Spring injects `Authentication` value in the parameter of the method

```
@GetMapping("/hello")
public String hello(Authentication a) {      ①
  return "Hello, " + a.getName() + "!";
}
```

① Spring Boot injects the current Authentication in the method parameter.

When calling the endpoint with a correct user, the response body contains the username. For example:

```
curl -u user:99ff79e3-8ca0-401c-a396-0a8625ab3bad http://localhost:8080/hello
Hello, user!
```

6.2.2 Using a holding strategy for asynchronous calls

It is easy to stick with the default strategy for managing the security context. In a lot of cases, it is the only thing you need. `MODE_THREADLOCAL` offers the ability to isolate the security context for each thread, and it makes the security context more natural to understand and manage. However, there are also cases in which this does not apply.

The situation gets more complicated if we have to deal with multiple threads per request. Look at what happens if you make the endpoint asynchronous. The thread that executes the method is no longer the same thread that serves the request. Think about an endpoint like the one presented in the next listing.

Listing 6.9 An `@Async` method served by a different thread

```
@GetMapping("/bye")
@Async                    ①
public void goodbye() {
  SecurityContext context = SecurityContextHolder.getContext();
  String username = context.getAuthentication().getName();
```

```
    // do something with the username
}
```

① Being @Async, the method is executed on a separate thread.

To enable the functionality of the `@Async` annotation, I have also created a configuration class and annotated it with `@EnableAsync`:

```
@Configuration
@EnableAsync
public class ProjectConfig {
}
```

NOTE Sometimes in articles or forums, the configuration annotations are placed over the main class. For example, you might find that certain examples use the `@EnableAsync` annotation directly over the main class. This approach is technically correct because we annotate the main class of a Spring Boot application with the `@SpringBootApplication` annotation, which includes the `@Configuration` characteristic. However, in a real-world application, we prefer to keep the responsibilities apart, and we never use the main class as a configuration class. To make things as clear as possible for the examples in this book, I prefer to keep these annotations over the `@Configuration` class, similar to how you'll find them in practical scenarios.

If you try the code as it is now, it will throw a `NullPointerException` on the line that gets the name from the authentication, which is

```
String username = context.getAuthentication().getName()
```

This is because the method now executes on another thread that does not inherit the security context. For this reason, the `Authorization` object is null and, in the context of the presented code, causing a `NullPointerException`. In this case, you could solve the problem by using the `MODE_INHERITABLETHREADLOCAL` strategy. This can be set either by calling the `SecurityContextHolder.setStrategyName()` method or by using the system property `spring.security.strategy`. By setting this strategy, the framework knows to copy the details of the original thread of the request to the newly created thread of the asynchronous method (figure 6.9).

Figure 6.9 When using the `MODE_INHERITABLETHREADLOCAL`, the framework copies the security context details from the original thread of the request to the security context of the new thread.

The next listing presents a way to set the security context management strategy by calling the `setStrategyName()` method.

Listing 6.10 Using `InitializingBean` to set `SecurityContextHolder` mode

```
@Configuration
@EnableAsync
public class ProjectConfig {

  @Bean
  public InitializingBean initializingBean() {
    return () -> SecurityContextHolder.setStrategyName(
      SecurityContextHolder.MODE_INHERITABLETHREADLOCAL);
  }
}
```

After calling the endpoint, you will observe that the security context is propagated correctly to the next thread by Spring. Additionally, `Authentication` is not null anymore.

NOTE This works only when the framework creates the thread itself (for example, in the case of an `@Async` method). If your code creates the thread, you will run into the same problem even with the `MODE_INHERITABLETHREADLOCAL` strategy. This happens because in this case, the framework does not know about the thread that your code creates. We'll discuss how to solve the problems of these cases in sections 6.2.4 and 6.2.5.

6.2.3 Using a holding strategy for standalone applications

If what you need is a security context shared by all the application threads, change the strategy to `MODE_GLOBAL` (figure 6.10). You would not use this strategy for a web server as it doesn't fit the general picture of the application. A backend web application manages independently the requests it receives, so it really makes more sense to have the security context separated per request instead of one context for all of them. Still, this can be a good use for a standalone application.

Figure 6.10 With `MODE_GLOBAL` used as the security context management strategy, all the threads access the same security context. This implies that these all have access to the same data and can change that information. Because of this, race conditions can occur, and you must take care of synchronization.

As the following code snippet shows, you can change the strategy like we did with `MODE_INHERITABLETHREADLOCAL`. You can use the method `SecurityContextHolder.setStrategyName()` or the system property `spring.security.strategy`:

```
@Bean
public InitializingBean initializingBean() {
  return () -> SecurityContextHolder.setStrategyName(
    SecurityContextHolder.MODE_GLOBAL);
}
```

Also, be aware that the `SecurityContext` is not thread safe. So with this strategy in which all the threads of the application can access the `SecurityContext` object, you need to take care of concurrent access.

6.2.4 Forwarding the security context with DelegatingSecurityContextRunnable

You have learned that you can manage the security context with three modes provided by Spring Security: `MODE_THREADLOCAL`, `MODE_INHERITEDTHREADLOCAL`, and `MODE_GLOBAL`. By default, the framework only makes sure to provide a security context for the request thread, and this security context is only accessible to that thread. However, the framework doesn't take care of newly created threads (for example, in case of an asynchronous method). Furthermore, you learned that for this situation, you must explicitly set a different mode for the management of the security context. But we still have a singularity: What happens when your code starts new threads without the framework knowing about them? Sometimes we name these *self-managed* threads because it is we who manage them, not the framework. In this section, we apply some utility tools provided by Spring Security that help you propagate the security context to newly created threads.

No specific strategy of the `SecurityContextHolder` offers you a solution to self-managed threads. In this case, you need to take care of the security context propagation. One solution for this is to use the `DelegatingSecurityContextRunnable` to decorate the tasks you want to execute on a separate thread. The `DelegatingSecurityContextRunnable` ex-

tends `Runnable`. You can use it following the execution of the task when there is no value expected. If you have a return value, then you can use the `Callable<T>` alternative, which is `DelegatingSecurityContextCallable<T>`. Both classes represent tasks executed asynchronously, as any other `Runnable` or `Callable`. Moreover, these make sure to copy the current security context for the thread that executes the task. As figure 6.11 shows, these objects decorate the original tasks and copy the security context to the new threads.

Figure 6.11 `DelegatingSecurityContextCallable` is designed as a decorator of the `Callable` object. When building such an object, you provide the callable task that the application executes asynchronously. `DelegatingSecurityContextCallable` copies the details from the security context to the new thread and then executes the task.

The next listing presents the use of `DelegatingSecurityContextCallable`. Let's start by defining a simple endpoint method that declares a `Callable` object. The `Callable` task returns the username from the current security context.

Listing 6.11 Defining a `Callable` object and executing it as a task on a separate thread

```
@GetMapping("/ciao")
public String ciao() throws Exception {
  Callable<String> task = () -> {
    SecurityContext context = SecurityContextHolder.getContext();
    return context.getAuthentication().getName();
  };

  // Omitted code
}
```

We continue the example by submitting the task to an `ExecutorService`. The response of the execution is retrieved and returned as a response body by the endpoint.

Listing 6.12 Defining an `ExecutorService` and submitting the task

```
@GetMapping("/ciao")
public String ciao() throws Exception {
  Callable<String> task = () -> {
    SecurityContext context = SecurityContextHolder.getContext();
    return context.getAuthentication().getName();
  };
```

```
  ExecutorService e = Executors.newCachedThreadPool();
  try {
    return "Ciao, " + e.submit(task).get() + "!";
  } finally {
    e.shutdown();
  }
}
```

If you run the application as is, you get nothing more than a `NullPointerException`. Inside the newly created thread to run the callable task, the authentication does not exist anymore, and the security context is empty. To solve this problem, we decorate the task with `DelegatingSecurityContextCallable`, which provides the current context to the new thread, as shown in the following listing.

Listing 6.13 Running the task decorated by `DelegatingSecurityContextCallable`

```
@GetMapping("/ciao")
public String ciao() throws Exception {
  Callable<String> task = () -> {
    SecurityContext context = SecurityContextHolder.getContext();
    return context.getAuthentication().getName();
  };

  ExecutorService e = Executors.newCachedThreadPool();
  try {
    var contextTask = new DelegatingSecurityContextCallable<>(task);
    return "Ciao, " + e.submit(contextTask).get() + "!";
  } finally {
    e.shutdown();
  }
}
```

Calling the endpoint now, you can observe that Spring propagated the security context to the thread in which the tasks execute:

```
curl -u user:2eb3f2e8-debd-420c-9680-48159b2ff905 http://localhost:8080/ciao
```

The response body for this call is

```
Ciao, user!
```

6.2.5 Forwarding the security context with DelegatingSecurityContextExecutorService

When dealing with threads that our code starts without letting the framework know about them, we must manage propagation of the details from the security context to the next thread. In section 6.2.4, you applied a technique to copy the details from the security context by making use of the task itself. Spring Security provides some great utility classes such as `DelegatingSecurityContextRunnable` and `DelegatingSecurityContextCallable`. These classes decorate the tasks you execute asynchronously and also take the responsibility for copying the details from the security context so that your implementation can access those from the newly created thread. However, we have a second option to deal with the security context propagation to a new thread, and this is to manage propagation from the thread pool instead of from the task itself. In this section, you'll learn how to apply this technique by using more great utility classes provided by Spring Security.

An alternative to decorating tasks is to use a particular type of `Executor`. In the next example, you can observe that the task remains a simple `Callable<T>`, but the thread still manages the security context. The propagation of the security context happens because an implementation called `DelegatingSecurityContextExecutorService` decorates the `ExecutorService`. The `DelegatingSecurityContextExecutorService` also takes care of the security context propagation, as presented in figure 6.12.

Figure 6.12 `DelegatingSecurityContextExecutorService` decorates an `ExecutorService` and propagates the security context details to the next thread before submitting the task.

The code in the following listing shows how to use a `DelegatingSecurityContextExecutorService` to decorate an `ExecutorService` so that when you submit the task, it takes care of propagating the details of the security context.

Listing 6.14 Propagating the `SecurityContext`

```
@GetMapping("/hola")
public String hola() throws Exception {
  Callable<String> task = () -> {
    SecurityContext context = SecurityContextHolder.getContext();
    return context.getAuthentication().getName();
  };
```

```
    ExecutorService e = Executors.newCachedThreadPool();
    e = new DelegatingSecurityContextExecutorService(e);
    try {
      return "Hola, " + e.submit(task).get() + "!";
    } finally {
      e.shutdown();
    }
  }
```

Call the endpoint to test that the `DelegatingSecurityContextExecutorService` correctly delegated the security context:

```
curl -u user:5a5124cc-060d-40b1-8aad-753d3da28dca http://localhost:8080/hola
```

The response body for this call is

```
Hola, user!
```

NOTE Of the classes that are related to concurrency support for the security context, you should keep in mind the ones presented in table 6.1.

Spring offers various implementations of the utility classes that can be used in your application to manage the security context when creating your own threads. In section 6.2.4, you implemented `DelegatingSecurityContextCallable`. In this section, we use `DelegatingSecurityContextExecutorService`. If you need to implement security context propagation for a scheduled task, then you will be happy to hear that Spring Security also offers you a decorator named `DelegatingSecurityContextScheduledExecutorService`. This mechanism is similar to the `DelegatingSecurityContextExecutorService` presented in this section, with the difference that it decorates a `ScheduledExecutorService`, allowing you to work with scheduled tasks.

Additionally, for more flexibility, Spring Security offers you a more abstract version of a decorator called `DelegatingSecurityContextExecutor`. This class directly decorates an `Executor`, which is the most abstract contract of this hierarchy of thread pools. You can choose it for the design of your application when you want to be able to replace the implementation of the thread pool with any of the choices the language provides.

Table 6.1 Objects responsible for delegating the security context to a separate thread

Class	Description
`DelegatingSecurityContextExecutor`	Implements the `Executor` interface and is designed to decorate an `Executor` object with the capability of forwarding the security context to the threads created by its pool.
DelegatingSecurityContextExecutorService	Implements the ExecutorService interface and is designed to decorate an ExecutorService object with the capability of forwarding the security context to the threads created by its pool.
`DelegatingSecurityContextScheduledExecutorService`	Implements the `ScheduledExecutorService` interface and is designed to decorate a `ScheduledExecutorService` object with the capability of forwarding the security context to the threads created by its pool.
DelegatingSecurityContextRunnable	Implements the Runnable interface and represents a task that is executed on a different thread without returning a response. Above a normal Runnable , it can also propagate a security context to use on the new thread.

`DelegatingSecurityContextCallable`	Implements the `Callable` interface and represents a task that is executed on a different thread and that will eventually return a response. Above a normal `Callable`, it can also propagate a security context to use on the new thread.

6.3 Understanding HTTP Basic and form-based login authentications

Up to now, we've only used HTTP Basic as the authentication method, but throughout this book, you'll learn that there are other possibilities as well. The HTTP Basic authentication method is simple, which makes it an excellent choice for examples and demonstration purposes or proof of concept. But for the same reason, it might not fit all the real-world scenarios that you'll need to implement.

In this section, you'll learn more configurations related to HTTP Basic. In addition, we'll talk about a new authentication method called `formLogin`. In the rest of this book, we'll discuss other authentication methods, which match different kinds of architectures well. We'll compare them so that you can understand the best practices, as well as the anti-patterns for authentication.

6.3.1 Using and configuring HTTP Basic

You are aware that HTTP Basic is the default authentication method, and we have seen the way it works in various examples in chapter 3. In this section, we add more details regarding the configuration of this authentication method.

For theoretical scenarios, the defaults that the HTTP Basic authentication comes with are great. However, in a more complex application, you might find the need to customize some of these settings. For example, you might want to implement a specific logic for the case in which the authentication process fails. You might even need to set some values on the response sent back to the client in this case. Let's consider these cases with practical examples to understand how you can implement this. I want to point out again how you can set this method explicitly, as shown in the following listing. You can find this example in the project ssia-ch6-ex3.

Listing 6.15 Setting the HTTP Basic authentication method

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    return http.build();
  }
}
```

You can call the `httpBasic()` method of the `HttpSecurity` instance with a parameter of `Customizer` type. This parameter allows you to set up some configurations related to the authentication method, for example, the realm name, as shown in listing 6.16. You can think about the realm as a protection space that uses a specific authentication method. For a complete description, refer to RFC 2617 at **https://tools.ietf.org/html/rfc2617**.

Listing 6.16 Configuring the realm name for the response of failed authentications

```
@Bean
public SecurityFilterChain configure(HttpSecurity http)
  throws Exception {

  http.httpBasic(c -> {
    c.realmName("OTHER");
    c.authenticationEntryPoint(new CustomEntryPoint());
  });

  http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

  return http.build();
}
```

Listing 6.16 presents an example of changing the realm name. The lambda expression used is, in fact, an object of type `Customizer<HttpBasicConfigurer<HttpSecurity>>`. The parameter of type `HttpBasicConfigurer<HttpSecurity>` allows us to call the `realmName()` method to rename the realm. You can use cURL with the `-v` flag to get a wordy

HTTP response in which the realm name is indeed changed. However, note that you'll find the `WWW-Authenticate` header in the response only when the HTTP response status is 401 Unauthorized and not when the HTTP response status is 200 OK. Here's the call to cURL:

```
curl -v http://localhost:8080/hello
```

The response of the call is

```
/
...
< WWW-Authenticate: Basic realm="OTHER"
...
```

In addition, by using a `Customizer`, we can customize the response for a failed authentication. You need to do this if the client of your system expects something specific in the response in the case of a failed authentication. You might need to add or remove one or more headers. Or you can have some logic that filters the body to make sure that the application doesn't expose any sensitive data to the client.

NOTE Always exercise caution about the data that you expose outside of the system. One of the most common mistakes (which is also part of the OWASP top ten vulnerabilities; see **https://owasp.org/www-project-top-ten/**) is exposing sensitive data. Working with the details that the application sends to the client for a failed authentication is always a point of risk for revealing confidential information.

To customize the response for a failed authentication, we can implement an `AuthenticationEntryPoint`. Its `commence()` method receives the `HttpServletRequest`, the `HttpServletResponse`, and the `AuthenticationException` that cause the authentication to fail. Listing 6.17 demonstrates a way to implement the `AuthenticationEntryPoint`, which adds a header to the response and sets the HTTP status to 401 Unauthorized.

NOTE It's a little bit ambiguous that the name of the `AuthenticationEntryPoint` interface doesn't reflect its usage on authentication failure. In the Spring Security architecture, this is used directly by a component called `ExceptionTranslationManager`, which handles any `AccessDeniedException` and `AuthenticationException` thrown within the filter chain. You can view the `ExceptionTranslationManager` as a bridge between Java exceptions and HTTP responses.

Listing 6.17 Implementing an `AuthenticationEntryPoint`

```
public class CustomEntryPoint
  implements AuthenticationEntryPoint {

  @Override
  public void commence(
    HttpServletRequest httpServletRequest,
    HttpServletResponse httpServletResponse,
    AuthenticationException e)
      throws IOException, ServletException {

      httpServletResponse
        .addHeader("message", "Luke, I am your father!");
      httpServletResponse
        .sendError(HttpStatus.UNAUTHORIZED.value());

    }
}
```

You can then register the `CustomEntryPoint` with the HTTP Basic method in the configuration class. The following listing presents the configuration class for the custom entry point.

Listing 6.18 Setting the custom `AuthenticationEntryPoint`

```
@Bean
public SecurityFilterChain configure(HttpSecurity http)
  throws Exception {

  http.httpBasic(c -> {
    c.realmName("OTHER");
    c.authenticationEntryPoint(new CustomEntryPoint());
  });

  http.authorizeHttpRequests().anyRequest().authenticated();

  return http.build();
}
```

If you now make a call to an endpoint so that the authentication fails, you should find the newly added header in the response:

```
curl -v http://localhost:8080/hello
```

The response of the call is

```
...
< HTTP/1.1 401
< Set-Cookie: JSESSIONID=459BAFA7E0E6246A463AD19B07569C7B; Path=/; HttpOnly
< message: Luke, I am your father!
...
```

6.3.2 Implementing authentication with form-based login

When developing a web application, you would probably like to present a user-friendly login form where the users can input their credentials. Furthermore, you might like your authenticated users to be able to surf through the web pages after they logged in and to be able to log out. For a small web application, you can take advantage of the form-based login method. In this section, you learn to apply and configure this authentication method for your application. To achieve this, we write a small web application that uses form-based login. Figure 6.13 describes the flow we'll implement. The examples in this section are part of the project ssia-ch6-ex4.

NOTE I link this method to a small web application because this way, we use a server-side session for managing the security context. For larger applications requiring horizontal scalability, using a server-side session for managing the security context is undesirable. We'll discuss these aspects in more detail in chapters 12 through 15 when dealing with OAuth 2.

Figure 6.13 Using form-based login. A user who has not yet been authenticated is sent to a form to log in with their credentials. After the application has verified their identity, they are directed to the application's main page.

To change the authentication method to form-based login, using the `HttpSecurity` object of the `SecurityFilterChain` bean instead of `httpBasic()`, call the `formLogin()` method of the `HttpSecurity` parameter. The following listing presents this change.

Listing 6.19 Changing the authentication method to a form-based login

```
@Configuration
public class ProjectConfig {
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
      throws Exception {

      http.formLogin(Customizer.withDefaults());

      http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

      return http.build();
    }
}
```

Even with this minimal configuration, Spring Security has already configured a login form, as well as a log-out page for your project. Starting the application and accessing it with the browser should redirect you to a login page (figure 6.14).

Figure 6.14 The default login page auto-configured by Spring Security when using the `formLogin()` method

You can log in using the default provided credentials as long as you do not register your `UserDetailsService`. These are, as we learned in chapter 2, username `user` and a UUID password that is printed in the console when the application starts. Because there is no other page defined, you are redirected to a default error page after a successful login. The application relies on the same architecture for authentication that we encountered in previous examples. So, as shown in figure 6.14, you need to implement a controller for the application homepage. The difference is that instead of having a simple JSON-formatted response, we want the endpoint to return HTML that can be interpreted by the browser as our web page. Because of this, we choose to stick to the Spring MVC flow and have the view rendered from a file after the execution of the action defined in the controller. Figure 6.15 presents the Spring MVC flow for rendering the homepage of the application.

Figure 6.15 A simple representation of the Spring MVC flow. The dispatcher finds the controller action associated with the given path (/home in this case). After executing the controller action, the view is rendered, and the response is sent back to the client.

To add a simple page to the application, you first must create an HTML file in the resources/static folder of the project. I call this file home.html. Inside it, type some text that you will be able to find afterward in the browser. You can just add a heading (for ex-

ample, `<h1>Welcome</h1>`). After creating the HTML page, a controller needs to define the mapping from the path to the view. The following listing presents the definition of the action method for the home.html page in the controller class.

Listing 6.20 Defining the action method of the controller for the home.html page

```
@Controller
public class HelloController {

  @GetMapping("/home")
  public String home() {
    return "home.html";
  }
}
```

Keep in mind that it is not a `@RestController` but a simple `@Controller`. Because of this, Spring does not send the value returned by the method in the HTTP response. Instead, it finds and renders the view with the name home.html.

Trying to access the /home path now, you are first asked if you want to log in. After a successful login, you are redirected to the homepage, where the welcome message appears. You can now access the /logout path, and this should redirect you to a log-out page (figure 6.16).

Figure 6.16 The log-out page configured by Spring Security for the form-based login authentication method

After attempting to access a path without being logged in, the user is automatically redirected to the login page. After a successful login, the application redirects the user back to the path they tried to originally access. If that path does not exist, the application displays a default error page. The `formLogin()` method returns an object of type `FormLoginConfigurer<HttpSecurity>`, which allows us to work on customizations. For example, you can do this by calling the `defaultSuccessUrl()` method, as shown in the following listing.

Listing 6.21 Setting a default success URL for the login form

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {
```

```
    http.formLogin(c -> c.defaultSuccessUrl("/home", true));

    http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

    return http.build();
  }
```

If you need to go even more into depth with this, using the `AuthenticationSuccessHandler` and `AuthenticationFailureHandler` objects offers a more detailed customization approach. These interfaces let you implement an object through which you can apply the logic executed for authentication. If you want to customize the logic for successful authentication, you can define an `AuthenticationSuccessHandler`. The `onAuthenticationSuccess()` method receives the servlet request, servlet response, and the `Authentication` object as parameters. In the next listing, you'll find an example of implementing the `onAuthenticationSuccess()` method to make different redirects depending on the granted authorities of the logged-in user.

Listing 6.22 Implementing an `AuthenticationSuccessHandler`

```
@Component
public class CustomAuthenticationSuccessHandler
  implements AuthenticationSuccessHandler {

  @Override
  public void onAuthenticationSuccess(
    HttpServletRequest httpServletRequest,
    HttpServletResponse httpServletResponse,
    Authentication authentication)
      throws IOException {

      var authorities = authentication.getAuthorities();

      var auth =
            authorities.stream()
              .filter(a -> a.getAuthority().equals("read"))
              .findFirst();          ①

      if (auth.isPresent()) {          ②
        httpServletResponse
          .sendRedirect("/home");
      } else {
        httpServletResponse
```

```
        .sendRedirect("/error");
    }
  }
}
```

① Returns an empty Optional object if the “read” authority doesn’t exist

② If the “read” authority exists, it redirects to /home.

There are situations in practical scenarios when a client expects a certain format of the response in case of failed authentication. They may expect a different HTTP status code than 401 Unauthorized or additional information in the body of the response. The most typical case I have found in applications involves sending a *request identifier*. This request identifier has a unique value used to trace back the request among multiple systems, and the application can send it in the body of the response in case of failed authentication. Another situation is when you want to sanitize the response to make sure that the application doesn’t expose sensitive data outside of the system. You might want to define custom logic for failed authentication simply by logging the event for further investigation.

If you would like to customize the logic that the application executes when authentication fails, you can do this similarly with an `AuthenticationFailureHandler` implementation. For example, if you want to add a specific header for any failed authentication, you could do something like what is shown in listing 6.23. You could, of course, implement any logic here as well. For the `AuthenticationFailureHandler`, `onAuthenticationFailure()` receives the request, response, and the `Authentication` object.

Listing 6.23 Implementing an `AuthenticationFailureHandler`

```
@Component
public class CustomAuthenticationFailureHandler
  implements AuthenticationFailureHandler {

  @Override
  public void onAuthenticationFailure(
    HttpServletRequest httpServletRequest,
    HttpServletResponse httpServletResponse,
    AuthenticationException e)  {

    try {

      httpServletResponse.setHeader("failed",
```

```
        LocalDateTime.now().toString());
      httpServletResponse.sendRedirect("/error");

    } catch (IOException ex) {
      throw new RuntimeException(ex);
    }
  }
}
```

To use the two objects, you need to register them in the `securityFilterChain()` method on the `FormLoginConfigurer` object returned by the `formLogin()` method. The following listing shows how to do this.

Listing 6.24 Registering the handler objects in the configuration class

```
@Configuration
public class ProjectConfig {

  private final CustomAuthenticationSuccessHandler authenticationSuccessHandler;
  private final CustomAuthenticationFailureHandler authenticationFailureHandler;

// Omitted constructor

@Bean
public UserDetailsService uds() {
  var uds = new InMemoryUserDetailsManager();

  uds.createUser(
    User.withDefaultPasswordEncoder()
        .username("john")
        .password("12345")
        .authorities("read")
        .build()
  );

  uds.createUser(
    User.withDefaultPasswordEncoder()
        .username("bill")
        .password("12345")
        .authorities("write")
        .build()
  );

  return uds;
```

```
}

@Bean
public SecurityFilterChain configure(HttpSecurity http)
 throws Exception {

  http.formLogin(c ->
   c.successHandler(authenticationSuccessHandler)
    .failureHandler(authenticationFailureHandler)
  );

  http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

  return http.build();
 }
}
```

For now, if you try to access the /home path using HTTP Basic with the proper username and password, you are returned a response with the status HTTP 302 Found. This is how the application tells you that it is trying to do a redirect. Even if you have provided the right username and password, it won't consider these and will instead try to send you to the login form as requested by the `formLogin` method. You can, however, change the configuration to support both the HTTP Basic and the form-based login methods, like in the following listing.

Listing 6.25 Using form-based login and HTTP Basic together

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
 throws Exception {

 http.formLogin(c ->

   c.successHandler(authenticationSuccessHandler)
    .failureHandler(authenticationFailureHandler)
 );

 http.httpBasic(Customizer.withDefaults());

 http.authorizeHttpRequests(c -> c.anyRequest().authenticated());

 return http.build();
}
```

Accessing the /home path now works with both the form-based login and HTTP Basic authentication methods:

```
curl -u user:cdd430f6-8ebc-49a6-9769-b0f3ce571d19 http://localhost:8080/home
```

The response of the call is

```
<h1>Welcome</h1>
```

Summary

- The `AuthenticationProvider` is the component that allows you to implement custom authentication logic.
- When you implement custom authentication logic, it's a good practice to keep the responsibilities decoupled. For user management, the `AuthenticationProvider` delegates to a `UserDetailsService`, and for the responsibility of password validation, the `AuthenticationProvider` delegates to a `PasswordEncoder`.
- The `SecurityContext` keeps details about the authenticated entity after successful authentication.
- You can use three strategies to manage the security context: `MODE_THREADLOCAL`, `MODE_INHERITABLETHREADLOCAL`, and `MODE_GLOBAL`. Access from different threads to the security context details works differently, depending on the mode you choose.
- Remember that when using the shared-thread local mode, it's only applied for threads managed by Spring. The framework won't copy the security context for the threads that are not governed by it.
- Spring Security offers you great utility classes to manage the threads created by your code, about which the framework is now aware. To manage the `SecurityContext` for the threads you create, you can use
  - `DelegatingSecurityContextRunnable`
  - `DelegatingSecurityContextCallable`
  - `DelegatingSecurityContextExecutor`
- Spring Security autoconfigures a form for login and an option to log out with the form-based login authentication method, `formLogin()`. It is straightforward to use when developing small web applications.
- The `formLogin` authentication method is highly customizable. Moreover, you can use this type of authentication together with the HTTP Basic method.

Previous chapter

< 5 A web app's security begins with filters

Next chapter

Part 3. Configuring authorization >